

# オペレーティングシステム I 令和7年度 後期末試験

(2026.02.05 重村 哲至)

IE4

番 氏名

模範解答

注意：以下で「プロセス」と「スレッド」は同じ意味で用いられていることがある。

## 1 語句に関する問題

次の文章の空欄に最適な言葉を語群から記号で答えなさい。

(1 点 × 25 問 = 25 点)

ヒント：語群の言葉を全て 1 回使用します。

複数のスレッドが資源を共有して処理を進めるとき、スレッドの実行タイミングにより処理結果が間違ってしまうことがある。このときは、資源の利用について (1) が発生している。プログラム中の (1) が発生する可能性がある部分は (2) と呼ばれる。同じ資源を操作する (2) が、複数のスレッドによって同時に実行されないように (3) を行う必要がある。

シングルプロセッサシステムでは、(2) の実行中にスレッドが (4) ないように割り込みを禁止すれば、(2) を安全に実行できる。しかし、割り込みを禁止・許可する命令は (5) なので、この方法はカーネル内部でしか用いることができない。マルチプロセッサシステムの場合は割り込みを禁止するだけでは不十分であり、(6) のような特別な命令を使用する。

セマフォは内部にカウンタとスレッドの (7) を持つデータ型である。セマフォに対して、カウンタの値を増加させる (8) と減少させる (9) を行うことができる。カウンタの値が 0 のセマフォに (9) を行おうとしたスレッドは待ち状態になる。この待ちは (10) ではないので CPU を無駄遣いしない。

UNIX のメモリ共有には、共有メモリセグメントを作成する `shmget` と仮想メモリ空間へ接続する `shmat` 等の (11) を用いる方法がある。準備ができた後は、(11) を使用しないで高速にプロセス間でデータ交換ができる。しかし、(12) が組み込まれていないのでセマフォなどを併用する必要がある。

メッセージ通信において通信相手を指定する方式には、相手プロセスの PID などを用いる (13) 方式と、リンクなどの識別子などを用いる (14) 方式がある。また、メッセージ形式には種類を表す (15) を含むものと含まないものがある。(15) を含む形式を用いる場合、受信プロセスは (15) を指定して選択的にメッセージを受信することができる。

モニタは Java のクラスに似た (16) 型的一种である。

モニタ内部の (17) は、(18) の働きにより同時には一つしか実行されない。モニタ内部の (19) は、外部から直接に操作することはできない。signal, wait などの操作ができる (20) をモニタの内部にいくつでも宣言することができる。(21) を行ったスレッドは (20) の待ち行列に入りブロックする。

複数のスレッドが互いに資源の解放を待ち合い、永遠に処理が進まなくなる状態を (22) と呼ぶ。(22) の原因の中には (23) と循環待ちがある。(23) を防ぐには必要な資源を全て (24) すれば良い。循環待ちを防ぐには (25) に制約を設ける。

語群：

- (あ) P 操作, (い) TS(Test and Set) 命令,  
(う) V 操作, (え) wait 操作, (お) ガード,  
(か) クリティカルセクション, (き) システムコール,  
(く) タグ, (け) デッドロック,  
(こ) ビジーウェイト, (さ) プリエンプション,  
(し) 一括確保, (す) 確保順序,  
(せ) 確保待ち, (そ) 間接指定, (た) 競合,  
(ち) 資源 (データ), (つ) 条件変数, (て) 相互排除,  
(と) 直接指定, (な) 抽象データ型,  
(に) 手続き (メソッド), (ぬ) 同期機構,  
(ね) 特権命令, (の) 待ち行列

|      |     |      |     |      |     |
|------|-----|------|-----|------|-----|
| (1)  | (た) | (2)  | (か) | (3)  | (て) |
| (4)  | (さ) | (5)  | (ね) | (6)  | (い) |
| (7)  | (の) | (8)  | (う) | (9)  | (あ) |
| (10) | (こ) | (11) | (き) | (12) | (ぬ) |
| (13) | (と) | (14) | (そ) | (15) | (く) |
| (16) | (な) | (17) | (に) | (18) | (お) |
| (19) | (ち) | (20) | (つ) | (21) | (え) |
| (22) | (け) | (23) | (せ) | (24) | (し) |
| (25) | (す) |      |     |      |     |

## 2 相互排除

1. 次の TeC 風のアセンブリ言語で記述した、二つのスレッドで実行されるプログラムについて答えなさい。

```
// スレッド 1
...
(A) LD  G0, NUM
(B) ADD G0, #1
(C) ST  G0, NUM
...
// スレッド 2
...
(a) LD  G0, NUM
(b) SUB G0, #2
(c) ST  G0, NUM
...
// 共有変数
NUM DC 2 // NUMの初期値=2
```

上の ADD 命令と SUB 命令では、オペランドの定数は異なる値になっているので注意！！

- (a) NUM=2 のとき、スレッド 1、スレッド 2 の順に実行された後の、NUM の値を答えなさい。(3 点)

NUM の最終値 1

- (b) NUM=2 のとき、スレッド 2、スレッド 1 の順に実行された後の、NUM の値を答えなさい。(3 点)

NUM の最終値 1

- (c) NUM=2 のとき、スレッド 1 とスレッド 2 が相互排除なしに並列実行されたとき (どちらのスレッドが先に実行されるか、あるいは、どのようなタイミングで同時に実行されるか分からないとき)、実行後の NUM に格納される可能性のある値を全て答えなさい。(6 点)

NUM の最終値 0, 1, 3

- (d) 上の、(A)~(C) や (a)~(c) のようなプログラムの区間は何と呼ばれるか答えなさい。(3 点)

名称 クリティカルセクション

2. 次のような INC(Increment) 機械語命令が使用できるとします。以下の問いに答えなさい。

INC (Increment) 命令

INC M

メモリ上の変数 M の値を 1 増加させる。

- ① バスをロックする
- ②  $R \leftarrow [M]$
- ③ バスのロックを解除する
- ④  $R \leftarrow R + 1$
- ⑤ バスをロックする
- ⑥  $[M] \leftarrow R$
- ⑦ バスのロックを解除する

R は CPU 内部で、値を一時的に保持するレジスタ

- (a) この命令は前問のスレッド 1 と同様な処理を実行します。単純なシングルプロセッサシステムなら、この命令は必ずメモリの値を 1 増加させることができます。理由を説明しなさい。(5 点)

一般に機械語命令の実行は途中で割り込まれることはない。よって、必ず①から⑦までの処理が連続して実行され、途中でプリエンプションしてスレッド 2 が NUM をアクセスすることがないので、確実に値を 1 増加させることができる。

- (b) マルチプロセッサシステムでは正常な結果にならない可能性があります。マルチプロセッサシステムでも必ず正常な結果になるようにするには、INC 機械語命令の処理をどのように改良すればよいか答えなさい。(5 点)

途中でバスのロックを解除しているので、他のプロセッサが実行するスレッド 2 が NUM にアクセスする可能性がある。途中でバスのロックを解除しないように、③と⑤のステップを廃止する。

### 3 セマフォの応用

次は授業で紹介した C 言語風の仮想言語を用いてスレッド間でのメッセージ通信を実現したものです。メッセージは `int` 型の値を一つだけ運ぶことができます。

```
1 int      buf;           // バッファ
2 Semaphore empty = 1;    // バッファが空き
3 Semaphore full  = _(a); // バッファが満
4 void send(int msg) {    // メッセージ送信
5     P(_(b));            // P操作
6     buf = msg;
7     V(_(c));            // V操作
8 }
9 int receive() {         // メッセージ受信
10    P(_(d));            // P操作
11    int tmp = buf;
12    V(_(e));            // V操作
13    return tmp;
14 }
```

1. プログラム中の (a) から (e) に適切な記述を答えなさい。なお、セマフォの操作は `P(empty)` のように記述するものとします。(3 点 × 5 問 = 15 点)

|     |       |
|-----|-------|
| (a) | 0     |
| (b) | empty |
| (c) | full  |
| (d) | full  |
| (e) | empty |

2. `receive` ルーチンでは、メッセージを一時変数 `tmp` に格納した後で `return tmp`; しています。 `buf` の値を直接 `return buf`; で返してはいけない理由を答えなさい。(5 点)

12 行の `V` 操作により 5 行の `P` 操作でブロックしていたスレッドが起床し、6 行で `buf` を上書きする。13 行で `return` するより前に `buf` が上書きされる可能性があるので、12 行で `V` 操作するより前に `buf` の値を `tmp` にコピーしておく必要がある。

### 4 モニタの応用

次は授業で紹介した Java 言語風の仮想言語を用いてメッセージ通信のリンクを実装した例です。バッファにデータがある時に `send`、または、バッファにデータがない時に `receive` を実行すると、スレッドは条件変数の待ち行列に入ります。メッセージは `int` 型の値を一つだけ運ぶことができます。

```
monitor Link {
    int buf;           // メッセージのバッファ
    boolean busy;      // バッファにデータがある
    Condition empty;   // 条件変数
    Condition full;    // 条件変数
    // 初期化プログラム
    Link() {
        busy = false;
    }
    // メッセージ送信
    public void send(int msg) {
        if (_(a)) full.wait();
        buf = msg;
        busy = true;
        _(b);
    }
    // メッセージ受信
    public int receive() {
        if (!busy) _(c);
        int msg = buf;
        busy = _(d);
        _(e);
        return msg;
    }
}
```

- プログラム中の (a) から (e) に適切な記述を答えなさい。(3 点 × 5 問 = 15 点)

|     |                |
|-----|----------------|
| (a) | busy           |
| (b) | empty.signal() |
| (c) | empty.wait()   |
| (d) | false          |
| (e) | full.signal()  |

## 5 セマフォによる実行順の制御

以下の C 言語風のプログラムにおいて 3 つの関数 `procA()`, `procB()`, `procC()` は、別々のスレッドによって並行実行されます。また、`printf()` 関数は複数スレッドの環境でも、正常に動作するものとしてします。

```
Semaphore s1 = 0;
Semaphore s2 = 0;

// スレッド 1 により実行される
void procA() {
    P(s1);                // (1)
    printf("A-1\n");       // (2)
    V(s1);                // (3)
    P(s1);                // (4)
    printf("A-2\n");       // (5)
}

// スレッド 2 により実行される
void procB() {
    printf("B-1\n");       // (6)
    V(s1);                // (7)
    P(s1);                // (8)
    printf("B-2\n");       // (9)
    V(s2);                // (10)
    P(s2);                // (11)
    printf("B-3\n");       // (12)
    V(s1);                // (13)
    V(s2);                // (14)
}

// スレッド 3 により実行される
void procC() {
    P(s2);                // (15)
    printf("C-1\n");       // (16)
    V(s2);                // (17)
    P(s2);                // (18)
    printf("C-2\n");       // (19)
}
```

1. このプログラムの出力には 2 つの可能性が  
あります。両方を答えなさい。

(同一出力部分 5 点,

異なる出力部分 2 点 × 2 種類 = 4 点

合計 9 点)

| 出力例 1                                         | 出力例 2                                         |
|-----------------------------------------------|-----------------------------------------------|
| B-1<br>A-1<br>B-2<br>C-1<br>B-3<br>A-2<br>C-2 | B-1<br>A-1<br>B-2<br>C-1<br>B-3<br>C-2<br>A-2 |

2. セマフォの操作を 2 箇所を追加して、必ず  
最後の出力が C-2 になるようにします。第  
何行の次に、どのようなセマフォ操作を追加  
すべきか答えなさい。ただし、終了しないス  
レッドが無いようにすること。

(3 点 × 2 問 = 6 点)

| 記入例   | (10) 行の次に<br>P(s1);<br>を追加 |
|-------|----------------------------|
| 1 箇所目 | 5 行の次に<br>V(s1);<br>を追加    |
| 2 箇所目 | 13 行の次に<br>P(s1);<br>を追加   |